

Improving Mechanical Performance for Extreme Environments

Final Report

WSU Mechanics and Materials



Daniel Book;

Daniel Lee;

Douglas Takada;

CptS 423 Software Design Project II

Fall 2024

Instructor: Ananth Jillepalli

TABLE OF CONTENTS

I.	Introduction	4
II.	Team Members & Bios	4
III.	Project Requirements Specification	5
III.1.	Project Stakeholders	5
III.2.	Use Cases	5
III.3.	Functional Requirements	7
III.3.1.	Graphical User Interface	7
III.3.2.	Impurity Identification	8
III.3.3.	Statistical Analysis	8
III.4.	Non-Functional Requirements	9
IV.	Software Design - From Solution Approach	10
IV.1.	Architecture Design	10
IV.1.1.	Overview	10
IV.1.2.	Subsystem Decomposition	11
IV.1.2.1.	Image Segmentation	11
IV.1.2.1.1.	Description	11
IV.1.2.1.2.	Concepts and Algorithms Generated	11
IV.1.2.1.3.	Interface Description	11
IV.1.2.2.	Data Analysis	12
IV.1.2.2.1.	Description	12
IV.1.2.2.2.	Concepts and Algorithms	12
IV.1.2.2.3.	Interface Description	12
IV.1.2.3.	View/Controller	13
IV.1.2.3.1.	Description	13
IV.1.2.3.2.	Concepts and Algorithms	13
IV.1.2.3.3.	Interface Description	13
IV.2.	Data design	14
IV.3.	User Interface Design	14
V.	Test Case Specifications and Results	15
V.1.	Testing Overview	15
V.2.	Environment Requirements	17
V.3.	Test Results	17
VI.	Projects and Tools used	20
VII.	Description of Final Prototype	21

VIII. Product Delivery Status	23
IX. Conclusions and Future Work	24
IX.1. Limitations and Recommendations	24
IX.2. Future Work	24
X. Acknowledgements	24
XI. Glossary	24
XII. References	25
XIII. Appendix A – Team Information	26
XIV. Appendix B - Example Testing Strategy Reporting	27
XV. Appendix C - Project Management	27
XVI. Appendix D	30

I. Introduction

One of the many vital components to the space industry and defense is protection against high velocity impact threats. It is crucial to know precisely how much damage manufactured protective shields can withstand before they break. Being able to accurately determine when a protective shield will fail will help these ultrahard ceramics and refractory alloys to be used safely and effectively to protect humans and equipment from impact threats.

Our team's aim is to develop a program that will analyze images of protective shields that have been damaged due to circumstances found in extreme environments and then gather information about the shields. The main technique that we will be using is image segmentation.[1] This will allow us to format the images in a way that we will then be able to efficiently analyze them. By analyzing the damage and imperfections in the materials, we will learn which circumstances allow for the best performance. This information will also be used in the development of stronger and more durable materials.

This project is for the WSU Mechanics and Materials lab. Our client is Dr. Arezoo Zare, and our mentor is Dr. Ananth Jallepalli. Dr. Zare is a professor at WSU and a Mechanics and Materials researcher and leads the lab at WSU. Dr. AJ is a professor at WSU and is instructing our cohort of capstone projects. Contact information for Dr. AJ is ananth.jillepalli@wsu.edu and for Dr. Zare it is arezoo.zare@wsu.edu.

II. Team Members & Bios

Daniel Book is a computer science student interested in app design, data analysis, and microcontroller projects. His prior projects have included creating controllers for custom hardware and designing and implementing desktop applications for monitoring and controlling fuel-cell testing environments. For this project, his responsibilities include researching image segmentation methods for pore analysis, and designing the UI implementation for the app. The most notable segmentation methods that he researched and tested are watershed, edge-based segmentation, and Mask R-CNN segmentation.

Douglas Takada is a computer science student interested in cybersecurity, data science, and web development projects. His prior projects have included created building an ad-blocking service for his home network and working a mobile app for cougar safe rides. Douglas's skills include C/C++, Python, and R. For this project, his responsibilities include researching and implementing deep learning models for images of pores within materials. Douglas has investigated the METAAI Segment Anything model (SAM) as well as Fast SAM. [2][3] In addition to the models, he has investigated services that allow these models to run such as Google Collab, Kaggle, and Kamiak.

Daniel Lee is a computer science student interested in software programming, game design, and mathematics. His prior projects include a spreadsheet application and library manager application. His skills include C/C++, C#, Python and HTML. For this project, his responsibilities

include developing an image segmentation program for impurity analysis and data mining research articles for more data for the client to use.

III. Project Requirements Specification

III.1. Project Stakeholders

Our project stakeholders include Dr. Ananth Jallepalli, Dr. Zare, Daniel Lee, Daniel Book, and Douglas Takada. Dr. Ananth Jallepalli's needs include ensuring the WSUM team satisfies the requirements outlined in the course syllabus and ensure that we pass the capstone class. Dr. Zare's need is to have a complete or a prototype of this project at the end of the project timeline to have accessible materials and mechanical scientists to use. Finally, for Daniel Lee, Daniel Book, and Douglas Takada, their needs include satisfying the capstone class's requirements and contributing to the project for the project's duration.

III.2. Use Cases

Change Parameters

Pre-Condition	- On the main menu of User Interface
Post-Condition	- Updated parameters of the image segmentation (pore identification) algorithm - Updated parameters of statistical analysis - Updated parameters of machine learning algorithm - Display new changes
Purpose	- Achieve accurate pore identification - Achieve accurate pore measurements - Achieve accurate correlation coefficient
Related Requirements	- Downloaded python executable

Import Images from Specified Folder

Pre-Condition	- On main menu of the User Interface
Post-Condition	- Imported image files into the current runtime - Display uploaded photos within folder - Display photos that were unable to be uploaded

Purpose	- Allows for images to be grouped for analysis - Gives analytic components of the app access to all necessary resources
Related Requirements	- Downloaded python executable - Images are locally downloaded in a folder on user's computer

Specify Groups of Imported Images for Analysis

Pre-Condition	- On main menu of the User Interface - Images have already been imported
Post-Condition	- Groups of images have been created - Display uploaded photos - Display photos that were selected but unable to be uploaded
Purpose	- Allows for images to be grouped for analysis - Gives analytic components of the app access to all necessary resources
Related Requirements	- Downloaded python executable - Images are locally downloaded on user's computer

Run Analysis on Groups

Pre-Condition	- On main menu of the User Interface - Images have been successfully imported into Python program - Valid parameters were set for the group of images - Image groups have been created
Post-Condition	- Display image segmentation results - Display statistical analysis results - Display parameters used for calculations - Display calculated results
Purpose	- Check if correct parameters were applied - Check if the image segmentation correctly identified pores - Identify patterns and relationships between pore structure and breakage patterns

Related Requirements	- Downloaded python executable - Images are locally downloaded on user's computer
-----------------------------	--

Download Analytics to Computer

Pre-Condition	- On main menu of the User Interface - Images have been successfully imported into Python program - Valid parameters were set for group of images - Ran analysis on selected photos
Post-Condition	- File of image segmentation, statistical and calculated results along with parameters is downloaded onto computer
Purpose	- Having a simple file to easily share with others and analyze results further - Document that shows information to replicate previous analysis on images
Related Requirements	- Downloaded python executable - Images are locally downloaded on user's computer

III.3. Functional Requirements

III.3.1. Graphical User Interface

Parameter Interface

Description	This interface will allow users to change parameters for the analysis of the images. This interface will allow parameters to be changed for the image segmentation, and statistical analysis.
Source	Internal requirements among members of the team.
Priority	Priority level is medium. The ability to change parameters will increase the breadth of the applications use cases, but it is not a necessity. A further, and lower priority, development to this requirement would be to incorporate this ability into a GUI.

Select Images

Description	The ability to select images that the user would like to analyze. This will import the images for use by the python program.
Source	External requirements requested by our client Dr. Zare.
Priority	Priority level is high. The ability to select which images will be analyzed is essential to the functionality of this application. A fully integrated UI pop-up, however, is a low priority requirement that will make the application more user friendly but is not a necessity.

Download Results

Description	A button that will allow the user to download a file of the analysis.
Source	External requirements requested by our client Dr. Zare.
Priority	Priority level is medium for this functional requirement. It is a desired functionality that will provide exceptional use for the application; however, it is not an absolute necessity as some of the other requirements are.

III.3.2. Impurity Identification

Description	Being able to identify and correctly label the impurity of the material is crucial to calculating measurements and running statistical analysis on the data. Our approach is to use image segmentation to find these impurities. Segmented images will be available to the user for further use if needed.
Source	External requirements requested by our client Dr. Zare.
Priority	Priority level is high. This is an essential and required functionality.

III.3.3. Statistical Analysis

Description	Once we can accurately identify and measure the pores, we will run statistical analysis on the data to find any relationships between the pore structure and the failure point. Results will be visualized if possible.
Source	External requirements requested by our client Dr. Zare.

Priority	Priority level is high. This is essential and required functionality.
-----------------	---

III.4. Non-Functional Requirements

Ease-of-Use

The system should be simple enough for unfamiliar researchers to operate the program by selecting images for processing to obtain statistical feedback they want, without the need for any detailed instructions or tutorials

Self-Contained

The system shall be an enclosed program where there should be no need for the user to access or be redirected to outside resources or tools

Efficient

The system shall be an efficient alternative to manual calculations that relieves some of the burden off researchers to reduce the time and cost of the process, while compiling the data into a convenient and extensive resource

IV. Software Design - From Solution Approach

IV.1. Architecture Design

The purpose of this section is to illustrate the fundamental design structure of our image analysis project in collaboration with the Washington State University Material Science Research Laboratory. We will be focusing on the three main design elements: Architecture, Data, and User Interface. In Architecture Design, we will dive into the overall structure of our software through the different subsystems and give a more in-depth explanation of the top-level design view of our system [1]. Within this section we will also focus on the subsystem's purpose, functionality, and connected interfaces. Lastly, for both the data and User Interface design we will have detailed explanations and example illustrations on their usage and components in the project.

Additionally, this section aims to create a roadmap for both developers and stakeholders to understand the design choices and process flow as development for this project goes on, alongside allowing for comparisons to intended objectives and desired outcomes.

Our goal is to develop a program for materials researchers that will analyze images of the materials used to create protective shields that have been damaged due to circumstances found in extreme environments and then gather information about their breakage patterns. The main technique that we will be using to distill this information is image segmentation. This will allow us to format and process the images in such a way that we can efficiently take various measurements and further analyze them. By analyzing the damage as well as imperfections in the materials, we will be able to better learn which circumstances allow for the best performance. This information will ideally be used in the development of stronger and more durable materials.

The core functionality of our project is the automated segmentation and analysis of images with high accuracy, as this program will be compared with traditional manual calculations. The functionalities can be separated into three main components: Image Segmentation, Data Analysis, and a Viewer/Controller. For design methodology we decided on the Scrum Sprint method since we are using Python and MATLAB for development. Regarding the design of our data structures, there was no concrete determination regarding how it should be structured. Our current plan is to implement primarily local and temporary variables in Python and MATLAB. However, depending on the needs of the project this may change. While our current situation allows our program to be easily adjustable, future implications will have to be considered as we near certain project breakpoints, such as when integrating the UI and image database with the Python package PyInstaller.

IV.1.1. Overview

The WSUM team has chosen to use a sub-style of the Call and Return architecture, namely the Object-Oriented architecture Model (Pranathi) for this project. Since the project is implemented in python and MATLAB, the architecture model that conformed the most with python was the Object-Oriented model. Below is a component diagram that illustrates the overarching architecture of our program. This diagram's purpose is to first have a clear diagram that allows

our team to neatly implement each component of our system. In addition, it provides a representation of the component architecture that is decomposed in the next section. As seen in the diagram below, the Python UI will communicate with select system handlers. Some handler that the Python UI element would communicate with are the UI handler to set up the page layout, icons, and any other dynamic and static elements.

IV.1.2. Subsystem Decomposition

IV.1.2.1. Image Segmentation

IV.1.2.1.1. Description

The image segmentation component of this application will be responsible for all the image processing that this application will handle. Loaded images will all be segmented to allow efficient analysis of the individual pores [6]. Once segmented, labels will be applied to the segmented portions and data regarding the dimensions and aspects of the pores will be collected and stored with respect to their labels.

IV.1.2.1.2. Concepts and Algorithms Generated

The methods of image segmentation that we will be using are watershed and adaptive thresholding. Since we have two different use cases, segmenting pores and segmenting imperfections, we plan on using those two methods to segment the use cases respectively. For the pores, since each pore is likely to be sharing a border with another one, thresholding would not work as well. Once the threshold is applied many of the pores will meld together and the segments will be inaccurate. To account for this, we will be using the watershed approach. This algorithm will find peaks and valleys throughout the image based on the coloring, and once the valleys are determined they will begin to fill up with different colors of “water.” By using different colors to fill each valley, it will be easy to differentiate separate pores regardless of how close they are to one another. When it comes to more general imperfections in the images, thresholding will work much better than watershed. For imperfections, they very rarely share borders with each other, so by thresholding the image it will efficiently segment the individual imperfections without the risk of overlapping. Furthermore, the use of adaptive thresholding will base the thresholds off the local variance in color around the imperfections, so even if there is a large range in intensity throughout the image the individual segments will still be properly applied.

IV.1.2.1.3. Interface Description

Services provided

Service Name	Service Provided To	Description
Information Gathering	Data Analysis Component	When the images are segmented data regarding the dimensions and other aspects of the segments are acquired. This information

		will be used in the data analysis component.
Segmented Images	Data Analysis Component	The images will be segmented and supplied to the data analysis component for the user to view the images pre and post segmentation.

Services Required

Service Name	Service Provided From
Image Importing	View/Controller

IV.1.2.2. Data Analysis

IV.1.2.2.1. Description

The data analysis component is responsible for analyzing the data collected from image segmentation. Once data is collected, it will be formatted and manipulated to visualize what has been found and provide insightful meaning.

IV.1.2.2.2. Concepts and Algorithms

The analysis will use pandas, NumPy, and Matplotlib, three powerful for mathematical calculations, statistical analysis, and data visualization. The data will be primarily formatted using Pandas DataFrames[7]. This data structure allows the data to be put into a table that can be easily manipulated using a variety of operations. NumPy will be used for the mathematical portion of the analysis. We plan on using it to find the variance in the data and any other insight requested by the client. Once all the information is formatted and computed, Matplotlib will be used to display it such that it can be easily understood by the application user.

IV.1.2.2.3. Interface Description

Services Provided

Service Name	Service Provided To	Description
Data Visualization	View/Controller	Generates plots for each image that show the image pre and post segmentation, as well as the visualizations

		of the analyzed data corresponding to each image.
--	--	---

Services Required

Service Name	Service Provided From
Information Gathering	Image Segmentation Component
Segmented Images	Image Segmentation Component

IV.1.2.3. View/Controller

IV.1.2.3.1. Description

The View/Controller component will be the main interface between the user and the application. On top of being the UI, this component will also control the flow of the application, bringing together the data analysis and image segmentation components.

IV.1.2.3.2. Concepts and Algorithms

Both the application's view and controller will be powered by PyQt5, a Python library based on the popular C library Qt. PyQt5 will display any relevant graphics to the user and provide user interaction with the controller. The controller will use PyQt5's event and execution system to keep the application running and responsive to user inputs. It will also handle the flow of data throughout the runtime of the application. This includes importing images, storing data in models for global use, and exporting data and processed images. The view will handle displaying any relevant information to the user. This will include the current state of the program as it processes images and data and visualizations of the results.

IV.1.2.3.3. Interface Description

Services Provided

Service Name	Service Provided To	Description
Image Importing	Image Segmentation Component	The user can select a folder of images that will be imported into the program. These images will be processed and analyzed by the other components of the application.

Image Exporting	User	Once processed images and data visualizations are available from the data analysis component, the user can download these files for further use.
Image and Data Visualization	User	Once the processed images and data visualizations are available from the data analysis component, the controller will display these elements along with the rest of the view for quick insight to the user.

Services Required

Service Name	Service Provided From
Data Visualization	Data Analysis Component

IV.2. Data design

Our program utilizes a data structure called a DataFrame from the pandas Python library, which is a 2-dimensional data structure that can act as an array or table like Excel, set with rows and columns. This allows us to store the data measurements obtained from image segmentation (such as size and shape) and tabulate it in a visual and organized way. The filled DataFrame is then used for data manipulation purposes such as creating histograms that illustrate the distributions of each measurement type regarding the pores in each segmented image. However, its further use case can also extend to other features depending on the future needs of the client and our own design choices.

IV.3. User Interface Design

Our current user interface will be a simple design with a focus on ease-of-use and accessibility for the researchers. For now, it will be a simple command line with a menu screen (Image 2) which will list four options: parameters, select images, view results, and close. The parameters button will let the user adjust certain settings and thresholds of the image segmentation process to produce different or more desired results. The select images button will prompt the user to submit a file of images from their local file system to be processed for segmentation. The view results button will display each segmented image and histogram pair on the screen, allowing the

user to go through each set individually with buttons. Finally, the close button will shut down the program.

This is more of a placeholder user interface and is not the immediate focus of our team for the program, but instead a task whose attention is relegated more to the second semester of this project. In the future, we plan to implement a more visual and detailed GUI that better resembles a more intuitive application for researchers.

V. Test Case Specifications and Results

V.1. Testing Overview

Our team's goal is to develop a program that will analyze images of materials that have been damaged due to circumstances found in extreme environments. Our team's aim is to design this program to help automate the process of manual calculations that Material Scientists and Mechanical Engineers previously had endured. The program's final state is a Graphical User Interface (GUI) that allows Scientists and Engineers alike to easily input images, change parameters for analysis, and receive the calculated outputs. Our program's analysis will be at the micron level where we can analyze the materials' pores and impurities. Our program will have four primary functionalities that are crucial for testing. The first is the proper identification of each pore and impurity of the material. Our second functionality will be the proper measurement of impurities. The third functionality will be the correct identification of a pore or an impurity image input. Finally, the fourth and final functionality is a properly working GUI.

Since our program is written in Python, we will be using a combination of automated and manual tests to verify that key deliverables are being met. In addition, since this project has a component of Machine Learning (ML) we will also be testing these ML models for accuracy.

The objective of the tests in the project is for our team to gain confidence that the code we've written will consistently compile and run. In addition to the reliability of our code, these tests verify that our program will meet the desired behavior based on its respective inputs and fulfill the expected program deliverables. These tests will also ensure the longevity of our program by making sure that any future updates to the program will not diminish the quality of the program or its deliverables. Automated and manual testing will accomplish all these objectives.

Since Python has a vast resource of libraires and frameworks for testing, we will be using these for our automated tests. For our manual tests, we will need to create and document extensive manual testing procedures. Although most of our testing will be automated, some components of our program where automation testing will be incredibly difficult, we will be doing manual testing.

As described in the introduction project overview, our four primary functionalities we will be testing pore/impurity identification, pore/impurity measurement, accuracy of pore or impurity identification for correct analysis, and finally, the GUI functionality. Our testing process will deliver 3 major deliverables. The first are the automated tests such as unit tests, integration tests, and system tests that cover most of the functionalities described. The second is a

document containing the testing procedure including environment setup, operational instructions and expected results for all manual functional tests. The final deliverable is the Continuous Integration (CI) and Continuous Delivery (CD) using CircleCI[8][9].

Although we do not have a clear timeline for our tests, we would like to start with the simplest tests such as unit tests and make our way up to more higher-level tests until we complete all necessary tests to verify our program requirements.

The purpose of this section is to have a clear outline of how we intend to integrate tests into our application. It will cover general testing, deployment process, and our testing plans for each core functionality of our application. While this section covers our plans for how we will implement our tests, our timeline for this is undetermined. Our timeline is not yet clear due to our project still being in the research and building phase.

Our application will be tested using both CI and CD pipelines. The CI pipeline will focus more on the technical and research aspects of our algorithms, while the CD pipeline will focus on the front-end of the application where the users will interact. Since we are using a variety of image segmentation techniques in our application, we will be consistently testing and evaluating the performance of these algorithms as we integrate new code and updates. This process will involve updating a segmentation algorithm, running the algorithm on a set of testing images, and comparing the results with previous segmentation results. Since the comparisons must be made by humans to ensure the segmentation is done correctly, this testing process will not be automated. The front-end portion of the application will involve improving the user interface with the image segmentation algorithms. Once users start using these services, their feedback will be used to improve and redeploy the UI.

Backend Testing Process

1. **Research Segmentation Algorithms:** This process will begin by researching various segmentation algorithms and choosing one to evaluate. The algorithms we currently plan to test are watershed segmentation, edge-based segmentation [10], and Mask R-CNN [11].
2. **Initial Implementation:** Once an algorithm is chosen, a simple implementation of that algorithm will be developed.
3. **Tune and Run:** In order to produce the most accurate results, the algorithm's parameters will be modified to better fit the use case.
4. **Evaluate and Repeat:** Once the results have been collected, the developer will evaluate the results and pinpoint the parameters that make the biggest difference. Once the results are evaluated, step 3 and 4 will be repeated until the algorithm is tuned to achieve its best performance.

Frontend Testing Process

1. **Initial Prototype:** When the backend of the application is ready for use, an initial prototype of the application's UI will be made and available for users. This prototype will be made to the best of the developer's ability, but it will also have a lot of room for improvement.

2. **User Feedback:** Once the UI has been used consistently, users will provide feedback on what improvements can be made to increase the usability of the application.
3. **Improve and Repeat:** With the feedback kept in mind, the developers will improve the user-friendliness of the UI and make a new update. From there, users will continue to provide feedback to the developers, and the UI will continue to improve.

V.2. Environment Requirements

Our testing environment will consist of computers that can support Python code and image processing. The image processing is not too CPU intensive, but it is the bottleneck that determines which computers can test. The only absolute necessity for testing is the ability to run Python. For front-end testing, the ability to run image segmentation algorithms without significantly slowing down the operating system is also required. The need to run Python will not be the case for frontend testing since the application will be compiled to an exe file for easier use and access.

V.3. Test Results

Aspect Being Tested - GUI	Expected Result	Observed Result	Test Result	Test Case Requirements
Load Image Button	Upon button press, the user sees a file dialog, selects an image, and then sees the image loaded into the GUI	File dialog successfully opens, and the image is successfully displayed in the GUI	PASS	GUI is open
Set Scale Button	Upon button press, the screen grays except for the image and the user can click to specify the scale. When 'Enter' is pressed, the user can enter the scale length.	The user successfully enters the scaling mode and can enter the length.	PASS	Image has been loaded in
Crop Button	Upon button press, the screen grays except for the image and the user can drag crosshairs to specify the new image crop. When 'Enter' is pressed, the image crops to the new size.	The user is able to drag crosshairs to specify a new image crop, and the image successfully crops to that size.	PASS	Image has been loaded in
Parameter Specification	The user can specify Intensity Threshold, Disk Radius, Thresholding Type, Blur Type, and Minimum Particle Area using buttons on the left side of the GUI. Corresponding dialogs will open when the edit	The user opens the correct dialogs and the text displaying current settings updates accordingly.	PASS	GUI is open

	buttons are pressed, and the red text on the bottom of the screen will update accordingly.			
Run Segmentation Button	Upon button press, the program runs the segmentation script in the background with the specified parameters. The script is run in a separate thread so the GUI does not lag or halt.	The script successfully runs in the background without halting the GUI, and the parameters are correctly passed to allow the script to run correctly.	PASS	Image has been loaded in, and scale is set
View Segmented Image	After running segmentation, the user can view the segmented image by selecting "Segmented Image" in the dropdown menu on the bottom right of the GUI.	The user immediately sees the segmented image after the script runs when the correct dropdown mode is selected.	PASS	Segmentation script has successfully run
Data Visualization	After running segmentation, the user can view histograms of the resulting data by selecting "Histograms" in the dropdown menu on the bottom right of the GUI. The user can also specify what type of data is being plotted using the dropdown menu on the left side of the GUI.	The user immediately sees the plot update after the script runs when the correct dropdown mode is selected. The plot updates when the data type is changed.	PASS	Segmentation script has successfully run

Aspect Being Tested – Segmentation Script	Expected Result	Observed Result	Test Result	Test Case Requirements
Blur	If blur_option is 0, 1, 2, or 3, then the program will run the image(s) with average blur, gaussian blur, median blur, and bilateral filtering, respectively.	Each value correctly corresponds to the right blurring method.	PASS	Edit blur_option

Thresholding	If thresh_option is 0, 1, 2, 3, then the program will run the image(s) with global thresholding, adaptive mean thresholding, adaptive gaussian thresholding, and Otsu's binarization, respectively.	Each value correctly corresponds to the right thresholding method.	PASS	Edit thresh_option
Visualizations	If test is True then the blurred, thresh, and segmented images are created, otherwise just the segmented image is created.	Both options produce their proper output images and are located on the user's system.	PASS	Edit test
CSVs	After segmenting an image(s), two csv files are produced per image.	Both csv files are properly created and are located on the user's system.	PASS	Image(s) is segmented

Aspect Being Tested – Data Analysis	Expected Result	Observed Result	Test Result	Test Case Requirements
Fitting PDFs	Best fitted PDF is selected and plotted Infront of histogram	Best fitted pdf is plotted	PASS	Data is available for histogram plotting and PDF fitting
Fitting KDE	KDE is fitted to data with given bandwidth parameter optimized using k-fold testing	KDE is plotted and bandwidth is optimized to show detail and overall trend	PASS	Data is available for histogram and plotting KDE
Aggregate segmented data and plot histogram for comparison	Plot both the segmented data with manual calculations and	Histograms for original and new data is plotted	PASS	Both segmented data and original data is available for plotting and

	compare histograms			computing given summary datapoints
Testing best ML architecture/model to use	Given a variety of tests, determine the best ML model that is the best use case for predicting impurity script values	Displays best ML model based on given input	PASS	Enough data to present to ML models to avoid overfitting.

VI. Projects and Tools used

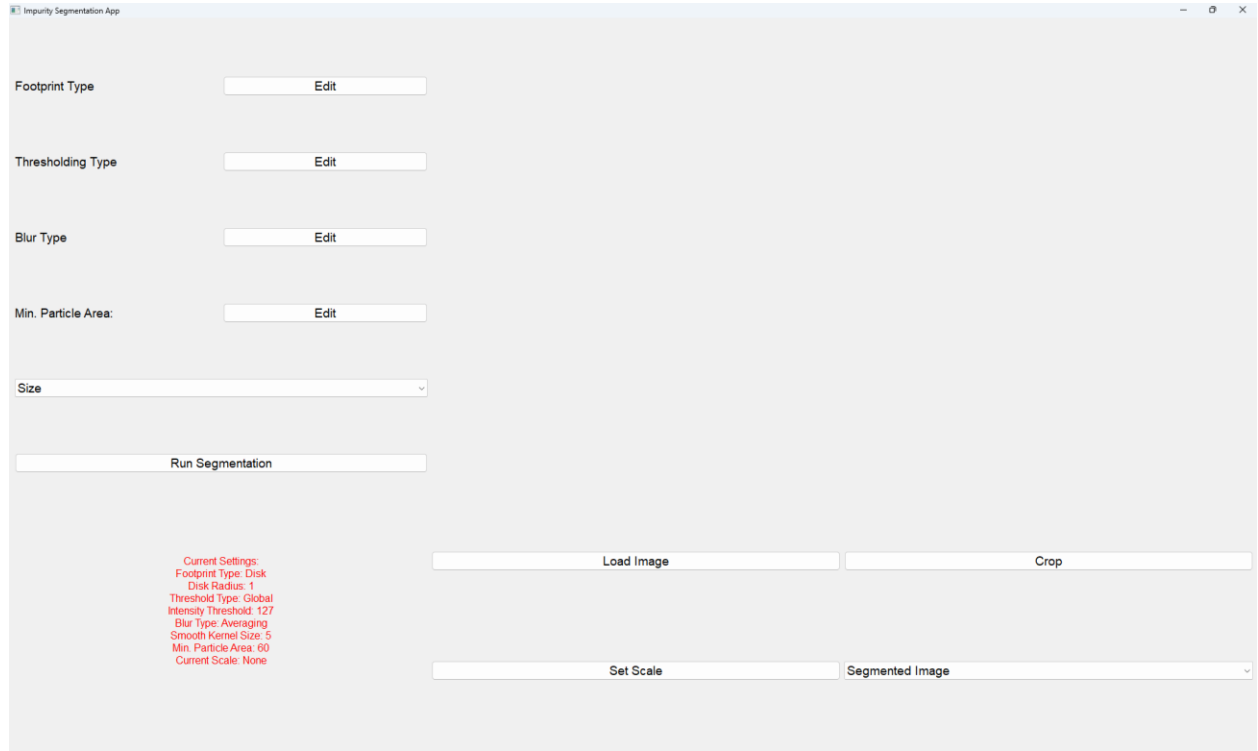
Tool/library/framework	Quick note on what it was for
Git	Software version control
Visual Studio Code	Used as an Integrated Developer Environment
Scikit-image	A python library used for image processing
matplotlib	A python library used to plot data
Numpy	A python library used for calculations
Pandas	A python library used for data manipulation and calculations
OpenCV	A python library used for computer vision tasks
Scipy	A python library used for statistical modeling of data
Fitter	A python library used for statistical modeling of data
PyQt5	A python library used for the GUI component of application
pyqtgraph	A python library for plots within the PyQt5 GUI

Languages Used in Project

Python

VII. Description of Final Prototype

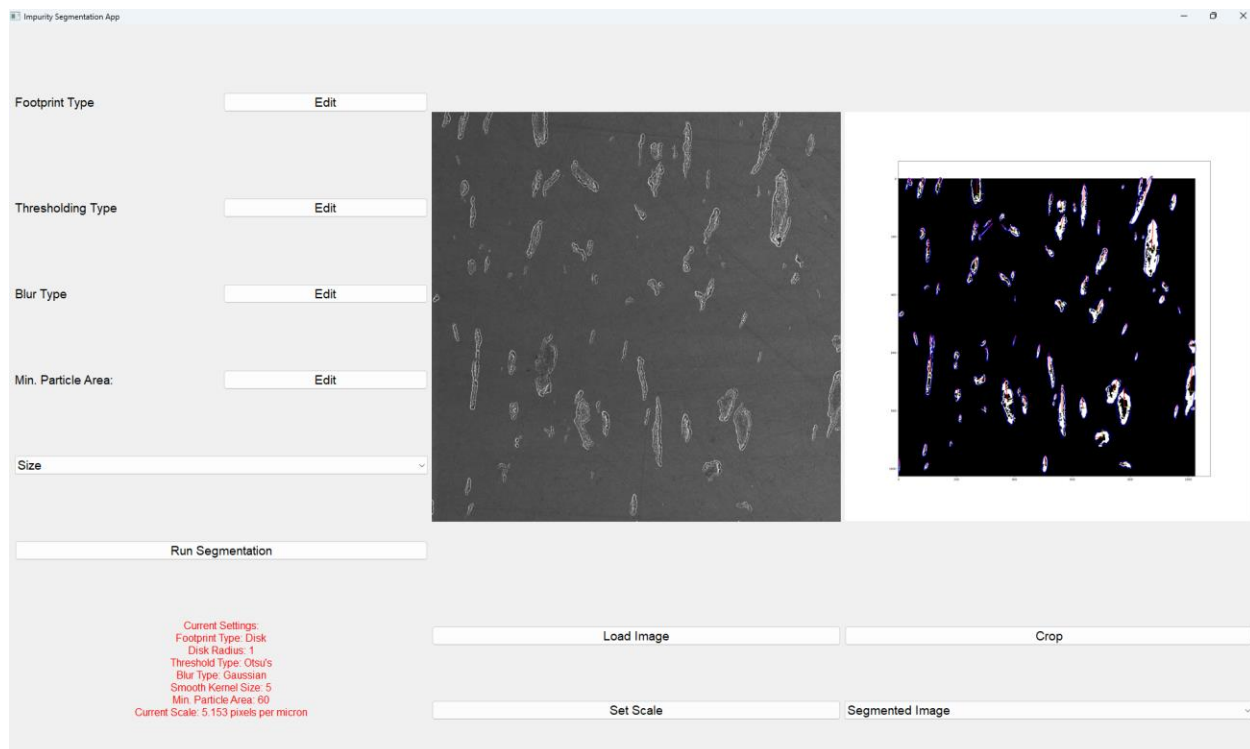
I. Graphical User Interface



In our final prototype, we have a functional GUI that allows users to visually access the components of our segmentation and analysis scripts. To start, users can load in impurity SEM images using the “Load Image” button. This will display the image on the screen, allowing users to set the scale and crop the image. The “Set Scale” button enables the user to click on the image, setting points to create a line across the visual scale. The user then enters the length of that line, which allows the segmentation script to convert pixel values into actual distances. The “Crop” button can then be used to cut off the visual scale, leaving only the impurities for the analysis. From there, users can refine the parameters using the menu on the left-hand side of the screen. Once the parameters are set, the “Run Segmentation” button will begin running the segmentation and analysis scripts using a separate thread. This allows the user to further interact with the program without having it halt. Using the dropdown menu on the bottom right, the user can then cycle between viewing the segmented image, histograms of the collected data, or the generated probability distribution functions.

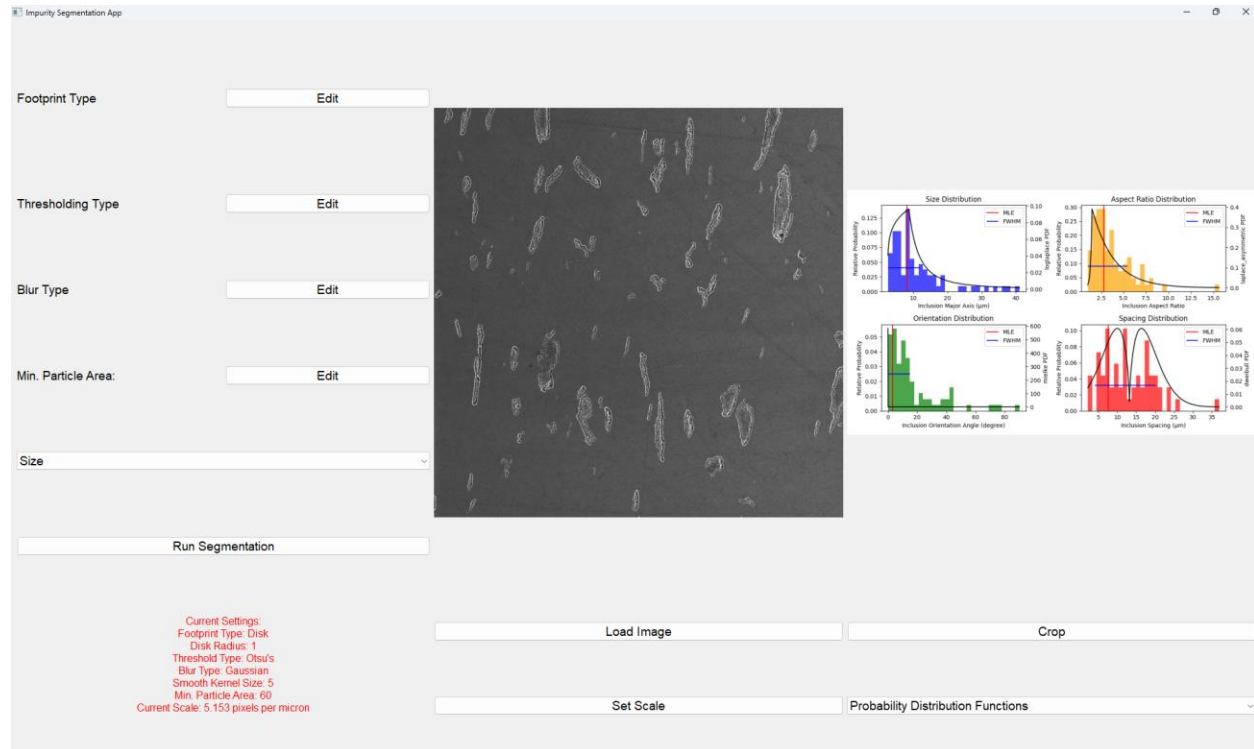
II. Segmentation Results

Improving Mechanical Performance for Extreme Environments – Final Report



After segmenting an image, the GUI then displays the segmented image to the side of the original image for the user to view and compare. This image shows the major and minor axis of each impurity and then fits an ellipse over each of them, which is saved to the user's system. All the data measurements for the segmented image, which is extracted by the program, are then aggregated into two CSV files (one for the size, aspect ratio, and orientation and one for the spacing) and saved to the user's system.

III. Data Analysis Results



This program analyzes impurity data from CSV files to model the distributions of size, aspect ratio, orientation, and spacing of inclusions in a material. It begins by loading necessary libraries for data manipulation, statistical analysis, and visualization. The program reads the data, cleans it, and extracts specific columns for analysis. For each feature, it fits the best probability distribution using the fitter library, identifying the distribution name and its parameters. Additionally, it calculates kernel density estimation (KDE) using `KernelDensity` from `sklearn`, optimizing the bandwidth with `GridSearchCV`. From the KDE, the program computes the maximum likelihood estimate (MLE) and the full width at half maximum (FWHM) to characterize the distributions further. It visualizes the data using `matplotlib` in a 2x2 grid, displaying histograms, fitted probability density functions (PDFs), and KDE plots annotated with MLE and FWHM. The results, including the fitted PDFs and distribution parameters, are saved as PNG images, and the computed MLE and FWHM values are appended to a CSV file. This comprehensive process ensures detailed statistical modeling and visualization of the impurity data.

VIII. Product Delivery Status

The project will be delivered to our client as an executable program at the end of the semester on December 6th, 2024. We have not gotten to the point of creating a demonstration or setup instructions for the executable.

Project location information:

- 1) Source repository
 - [WSUCptSCapstone-S24-F24/-wsum-pythonapps \(github.com\)](https://github.com/WSUCptSCapstone-S24-F24/-wsum-pythonapps)

- 2) Equipment storage location
 - Client-owned Onedrive File containing research data, images, etc.
- 3) Any other materials needed to rebuild your project from the ground up
 - In the source Repository there is a page titled README.md that has instructions on how to set up the application from the ground up.

IX. Conclusions and Future Work

IX.1. Limitations and Recommendations

Current problems may not be a good representative of the limitations and problems we may have later near the end of the project timeline. However, here are some of the challenges and limitations we are facing to get an idea of how the project is going. A primary problem we are trying to resolve is determining how to aggregate data in a way we can map features to the target features in the data set. Another issue is determining the accuracy of the image segmentation script as there is no concrete goal for the measurements it produces to aim for. Finally, the features of the GUI we make are subject to what the client desires and every update/demonstration to them may lead to newer insights and features.

IX.2. Future Work

Now that we have successfully created a GUI that allows users to segment impurity images, we can expand upon that functionality in a few ways. The primary objective is to allow the GUI to support batch segmentation. This means that users can upload entire folders of images to collect data among a larger sample. Another important objective is to have the program support pore segmentation. Although our team researched pore segmentation, we did not have the opportunity to include it in the GUI as a functional component.

X. Acknowledgements

We would like to express our deepest gratitude to Washington State University, whose resources and support made this research possible. We are particularly thankful to Dr. Arezoo Zare and Dr. Ananth Jallepalli for their insightful feedback and invaluable guidance throughout this work. We are also grateful to our families and friends for their patience and encouragement throughout the research process. Finally, we acknowledge the contributions of all participants who took part in this study, as their involvement was essential to its success.

XI. Glossary

Image Segmentation – An image processing technique to partition an image into multiple regions for the purpose of analysis.

Impurity – The breakage cracks and patterns within ceramics/metallic alloys as a result of dynamic compression of the material.

Python – A programming language used to develop software to perform specific functions.

GUI – Graphical User Interface. A visual component for users to interact with a program.

XII. References

- [1] “Image Segmentation,” MathWorks. <https://www.mathworks.com/discovery/image-segmentation.html#image-segmentation-with-matlab> (accessed Jan. 25, 2024).
- [2] P. S. Apr 12 and Read 2023 5 M., “How to Use the Segment Anything Model (SAM),” Roboflow Blog, Apr. 12, 2023. <https://blog.roboflow.com/how-to-use-segment-anything-model-sam/>
- [3] “segment-anything-fast/amg_example at main · pytorch-labs/segment-anything-fast,” GitHub. https://github.com/pytorch-labs/segment-anything-fast/tree/main/amg_example (accessed Apr. 19, 2024).
- [4] “Software Engineering | Architectural Design - GeeksforGeeks,” GeeksforGeeks, Jul. 23, 2018. <https://www.geeksforgeeks.org/software-engineering-architectural-design/>
- [5] Analytics Vidhya, “Step-by-Step Tutorial on Image Segmentation Techniques in Python,” *Analytics Vidhya*, May 06, 2019. <https://www.analyticsvidhya.com/blog/2019/04/introduction-image-segmentation-techniques-python/>
- [6] “OpenCV: Image Segmentation with Watershed Algorithm,” *docs.opencv.org*. https://docs.opencv.org/4.x/d3/db4/tutorial_py_watershed.html
- [7] “pandas.DataFrame — pandas 1.2.4 documentation,” *pandas.pydata.org*, 2023. <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html> (accessed Mar. 20, 2024).
- [8] “Configure a Python application on CircleCI - CircleCI,” *circleci.com*. <https://circleci.com/docs/language-python/> (accessed Mar. 22, 2024).
- [9] “CI vs CD: Difference between Continuous Integration and Continuous Delivery?,” BrowserStack. <https://www.browserstack.com/guide/difference-between-continuous-integration-and-continuous-delivery#:~:text=CI%20can%20be%20considered%20as> (accessed Mar. 22, 2024).
- [10] “Edge-Based Segmentation,” *Cloudinary*. <https://cloudinary.com/glossary/edge-based-segmentation>

Improving Mechanical Performance for Extreme Environments – Final Report

[11] E. Odemakinde, “Mask R-CNN: A Beginner’s Guide,” *viso.ai*, Mar. 19, 2021.

<https://viso.ai/deep-learning/mask-r-cnn/>

[12] OpenCV, “OpenCV: Image Thresholding,” *docs.opencv.org*.

https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html

[13] “skimage.measure — skimage 0.22.0 documentation,” *scikit-image.org*. <https://scikit-image.org/docs/stable/api/skimimage.measure.html> (accessed Mar. 15, 2024).

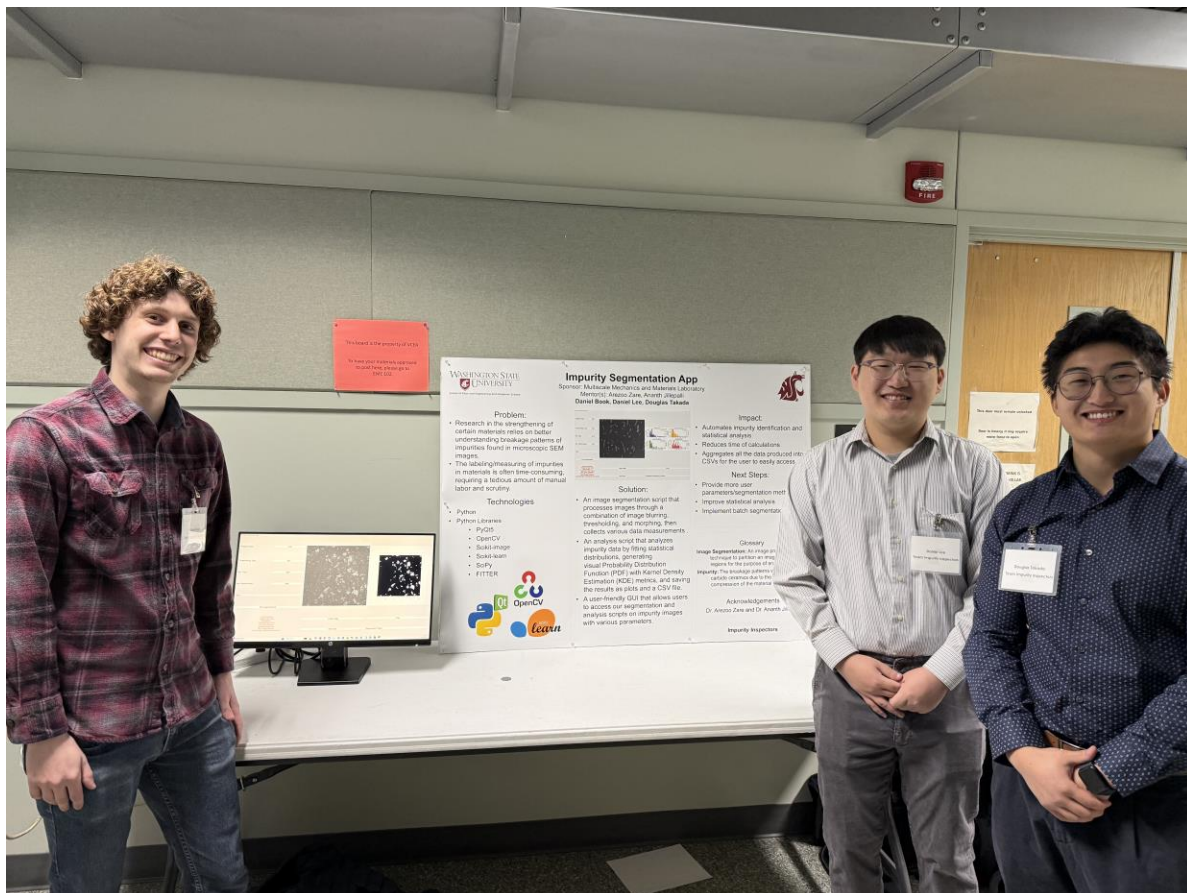
[14] “Watershed Segmentation - an overview | ScienceDirect Topics,” *Sciencedirect.com*, 2013.

<https://www.sciencedirect.com/topics/computer-science/watershed-segmentation>

[15] “Cluster Resources | High Performance Computing | Washington State University.”

<https://hpc.wsu.edu/kamiak-hpc/queue-list/> (accessed Apr. 19, 2024).

XIII. Appendix A – Team Information



Team Members: Daniel Book (left), Daniel Lee (middle), Douglas Takada (right)

XIV. Appendix B - Example Testing Strategy Reporting

Identify the requirements being tested

- GUI can select image files
- GUI can receive input on different parameters and change dynamically
- Statistical model can find relationships between the impurity size and spacing to the material compressive strength

XV. Appendix C - Project Management

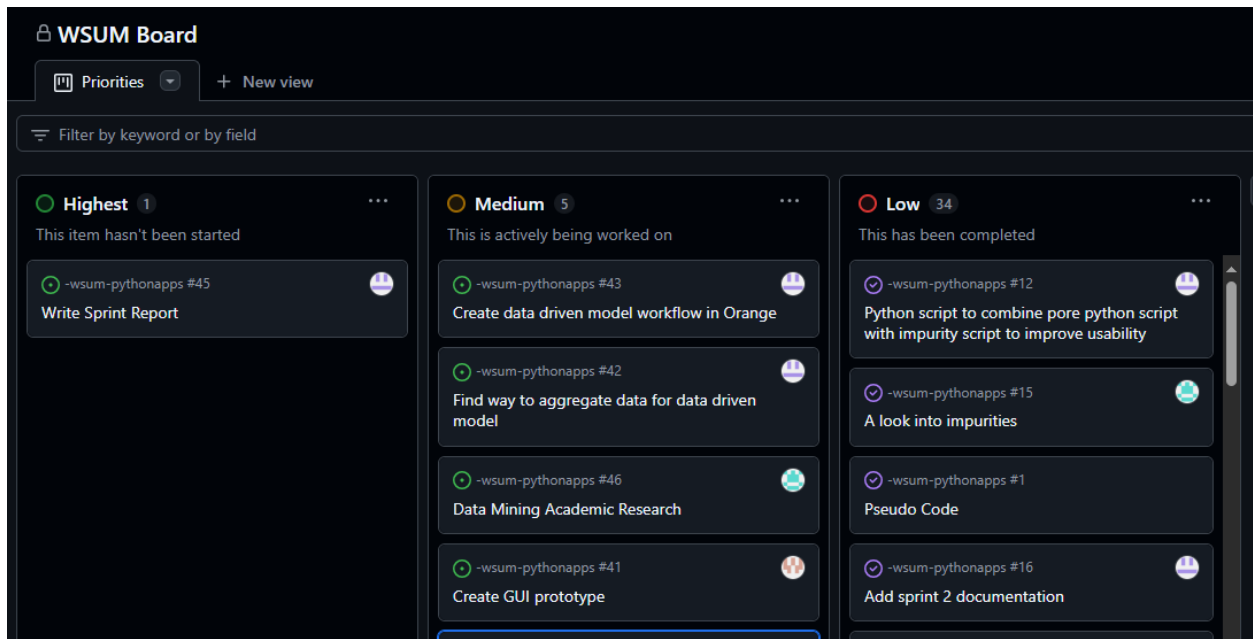
- Weekly meetings with the Client
 - Purpose: Go over the next steps in the project and resolve any misunderstandings or confusion.
 - Routine Agenda: Each team member goes over what they did for the week and the client gives input into what they should do next or change.
- Bi-weekly meetings with instructor
 - Purpose: To ensure we do not fall behind on our deadlines and GitHub commits.
 - Routine Agenda: Dr. AJ will give us feedback on what we are doing well and what we need to focus on. The rest of the time is for questions.
- Weekly meetings only with the team members
 - Purpose: To ensure we are on top of our work and discuss any needed tasks.
 - Routine Agenda: We talk about what our next steps are for the project, who will work on what, and any updates on each person's work.

The weekly client meetings were the most beneficial because they helped us to touch base with our client and discuss what needed to be done every week.

Planning documents:

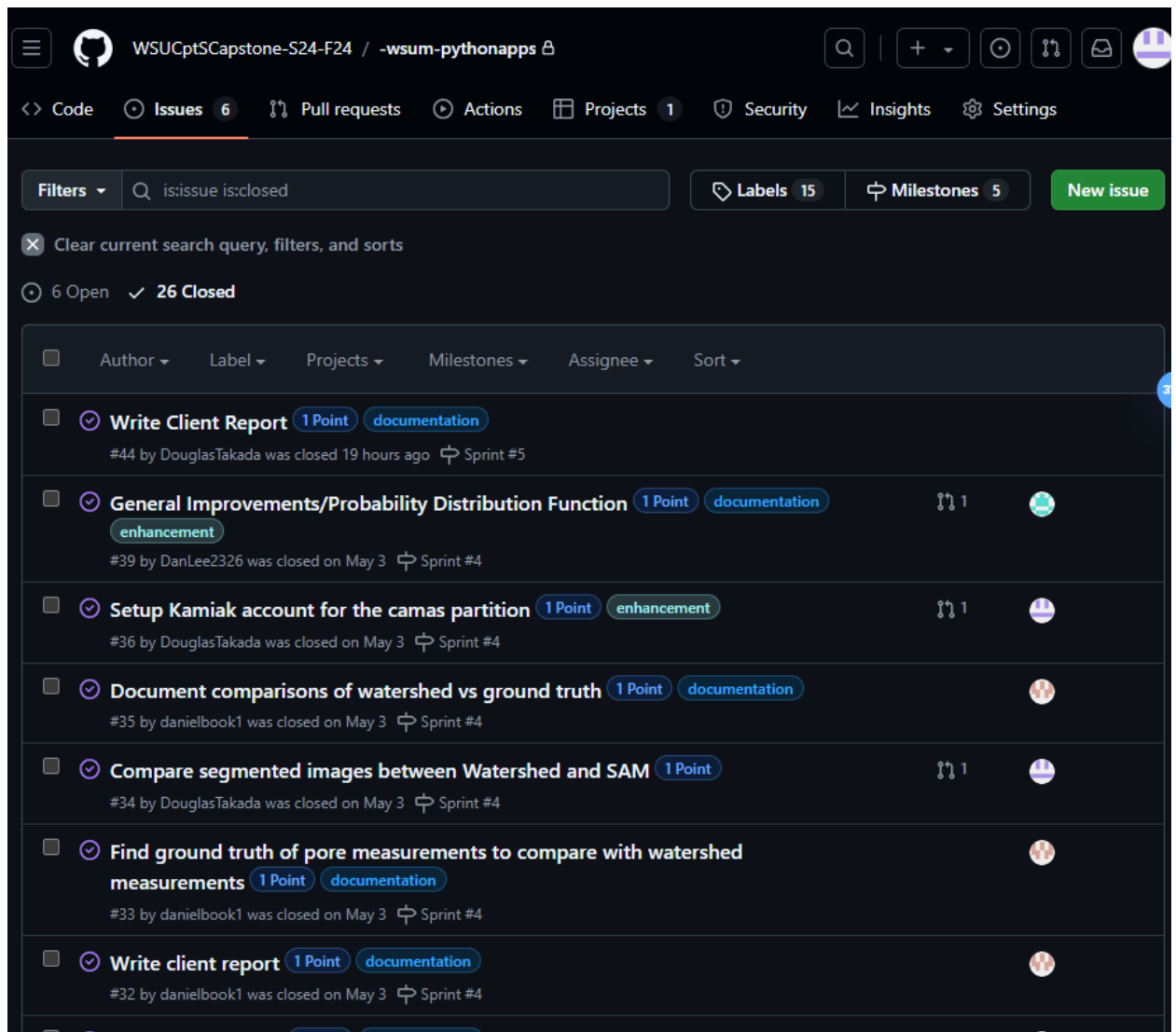
- GitHub projects

Improving Mechanical Performance for Extreme Environments – Final Report



GitHub project board page. Used for prioritizing issues for each sprint.

- GitHub issues



GitHub issues tab. Used to keep track of what we need to complete in each sprint.

- Notes on team tools used:
 - Discord
 - Teams
 - Zoom
 - Outlook
 - OneDrive

XVI. Appendix D

